

Lec2. IFT, Reward Models & Rejection Sampling

2026.10.02. Jeongyeon Kim

Table of Contents

Part1. Insturtion Tuning

Part2. Reward models

- Part2.5 Reward model Variants

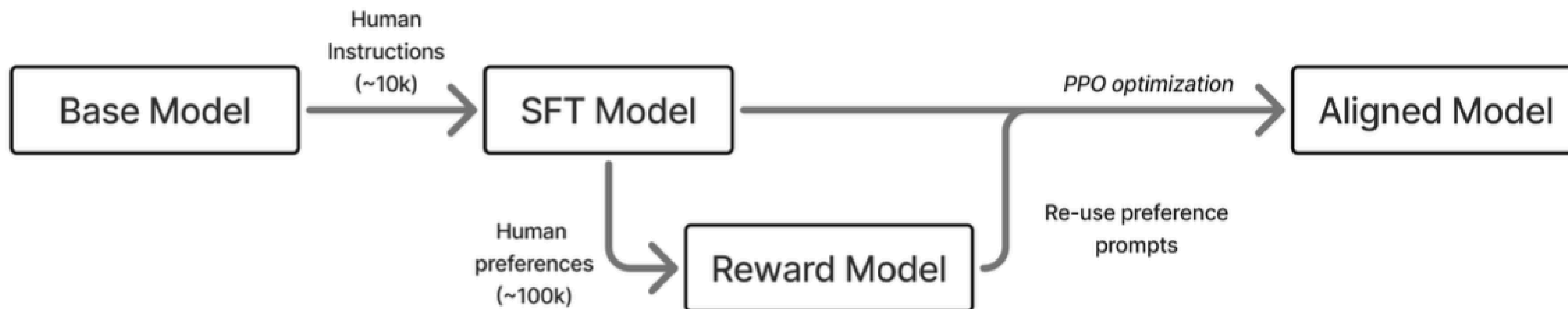
Part3. Rejection Sampling

Part1. Instruction tuning

Part 2. Reward models

LM's optimization & fine-tuning (Recall)

1. Pretraining : next-token cross entropy : learn general language ability
2. mid-training : final stage of pretraining (lr ↓ , data quality ↑) ... **start here!**
3. **SFT** : instruction-response pair : **learn how to response request !**
4. prepare Reward Model (supervised learning)
5. Optimization of RL : use Reward Model, **update SFT model**



The RLHF pipeline — instruction tuning is step 1.

Using completion mask: only the response tokens contribute to the loss, the prompt merely conditions. The model learns to respond , not just continue text.

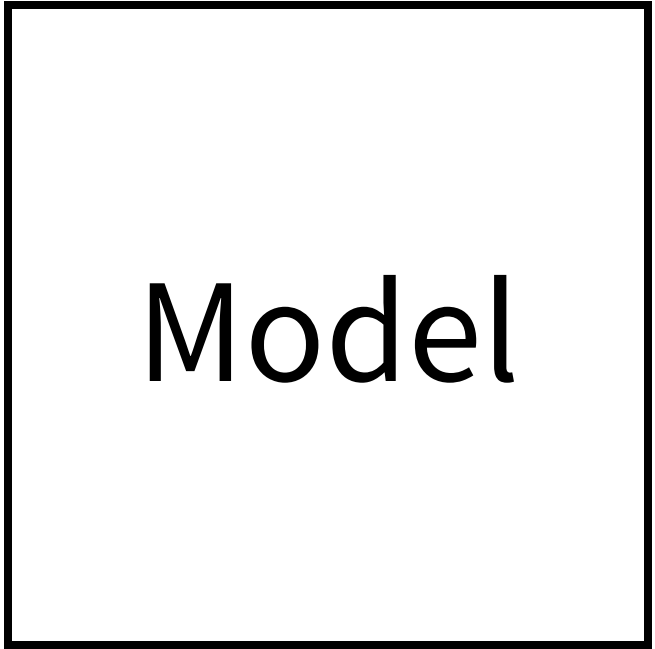
How to construct instruction-response pair data

Chat templates : the structure of instruction tuning

There is model & user!! Learn interaction!!

1. Who is speaking?

- **user** : human's messages
- **assistant** : model's responses



Model

2. What to generate?

- **system** : background instructions (persona, constraints)

Chat templates

chat template은 메시지 목록을 모델이 처리할 하나의 시퀀스로 변환

Use special tokens : role과 메시지 경계를 표시

multi-turn 대화에서는 이전 대화도 문맥에 포함

```
<|im_start|>system
You are a friendly chatbot who always
responds in the style of a pirate<|im_end|>
<|im_start|>user
How many helicopters can a human eat
in one sitting?<|im_end|>
<|im_start|>assistant
```

모델마다 학습한 special token과 대화 형식이 다르기 때문에, 해당 모델에 맞는 chat template을 사용해야 함

- Chat templates : 대화를 어떤 형식으로 변환할지 정한 규칙 ... not easily human-interpretable! 💀
- Jinja : 그 규칙을 반복문과 조건문으로 구현한 도구 ... 대안) open ai의 harmony

Insturction Tuning

- What does the model actually learn?
 - Prompt masking : loss는 assistant가 생성해야 하는 부분에 대해서만 계산
 - multi turn일 때는 Assistant 1에도 loss를 걸 수도 있고, Assistant 2에만 걸수도 있음
- Best practices for instruction tuning
 - data quality >> data quantity
 - data는 요즘 합성데이터로 보통 만드는 편
 - 100만개 정도면 충분, 당연히 데이터를 더 늘리면 성능 좋아질 수 있지만, marginal gain은 점점 작아짐
 - reasoning model에서는 SFT prompt 수가 줄었음 (open question)
 - 긴 reasoning 응답으로 전체 학습 토큰 수는 증가할 수 있음
 - prompt distrubution 중요! $P_{\text{sft}}(x) \approx P_{\text{downstream}}(x)$
- Training details
 - batch size, learning rate 작게
 - loss function : Cross entropy
- SFT data scaling의 근본적인 한계가 어디인지, 그리고 이러한 scaling이 RL과 어떻게 상호작용하는지는 밝혀지지 x

Building SFT data : Synthetic completions

1. 어떤 skills을 키울지 결정
2. 그 skill을 대표하는 좋은 seed prompts 확보
3. Strong LM으로 prompt를 synthetic하게 확장
4. Strong LM(3이랑 같아도 되고 달라도 됨)으로 completion 생성
5. filtering을 거쳐 대규모 SFT dataset 구축
6. SFT

The SFT design process

아래 2 작업을 병렬적으로 진행

1. 데이터 믹싱 : 이미 가지고 있는 데이터를 어떤 비율로 섞을 것인가
 - a. mixing ration 조정
 - b. +) 이 데이터 빼도 성능이 유지되는가?도 적극적으로 실험
2. 데이터 curation : 현재 데이터에 부족한 능력을 보완하기 위해 어떤 데이터를 새로 만들 것인가?
 - c. 우리 모델은 A 능력 모자라네? → 그 능력 target한 데이터 생성

SFT desing process는 아직 업계 기밀 ... ㅎ

Part2. Reward models

Part 2.5. Reward model variants

Reward Modeling — Learning a Preference Signal

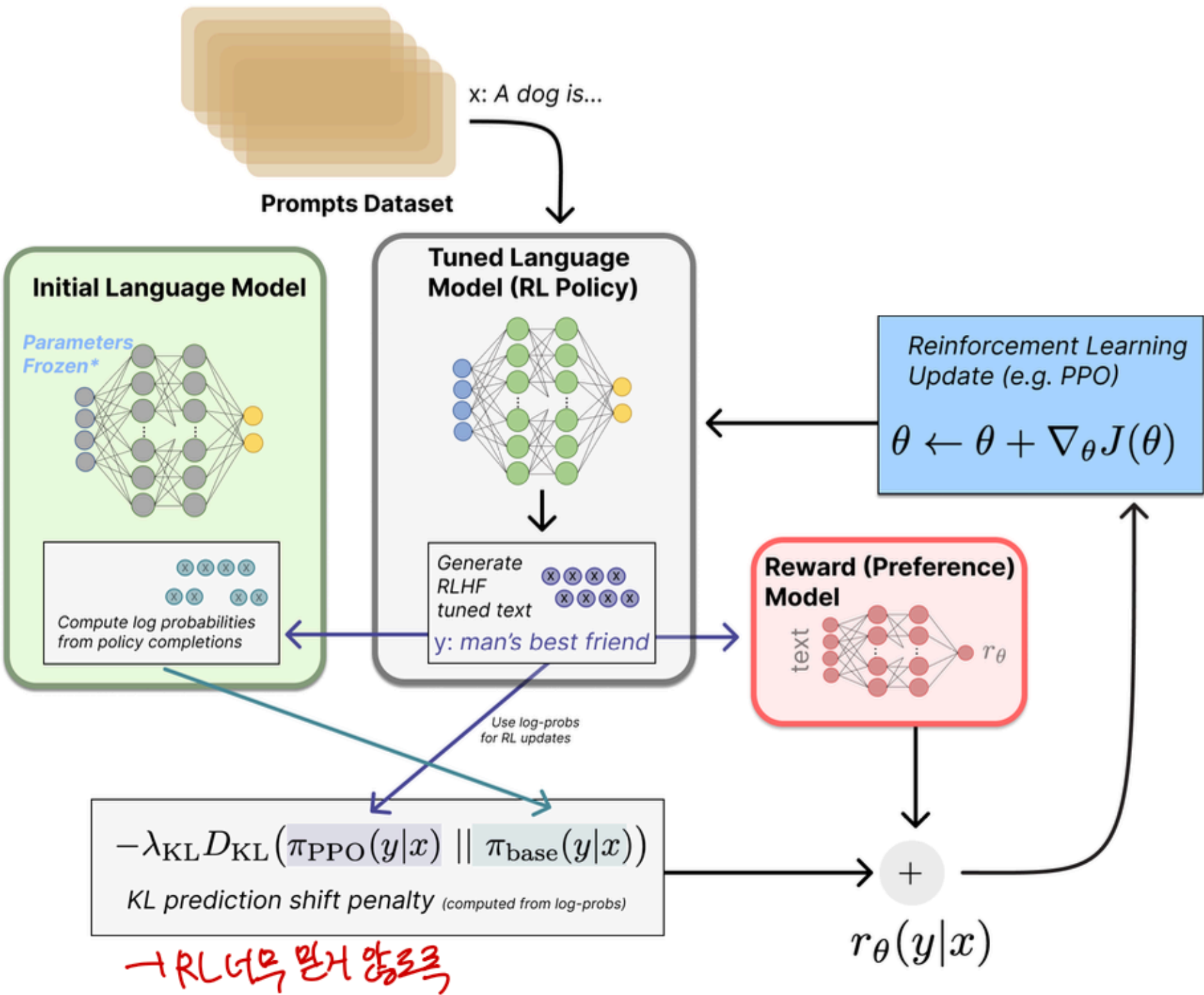
데이터 형식:

$$(x, y_c, y_r)$$

- y_c : chosen response
- y_r : rejected response

학습 목표

$$r_{\phi}(x, y_c) > r_{\phi}(x, y_r)$$



The reward model in RLHF plays the role of the environment's reward signal.

Bradley-Terry Model

두 응답의 점수를 선호 확률로 바꾸는 모델 only **score differences** matter

$$P(y_c \succ y_r \mid x) = \frac{\exp(r_c)}{\exp(r_c) + \exp(r_r)} = \sigma(r_c - r_r)$$

$$r_c = r_\phi(x, y_c), r_r = r_\phi(x, y_r)$$

Reward Model Loss — Maximum Likelihood

주어진 질문 x 에 대해, 선호된 응답 y_c 와 선호되지 않은 응답 y_r 의 선호 확률을 다음과 같이 정의한다.

$$P_{\phi}(y_c \succ y_r \mid x) = \sigma(r_{\phi}(x, y_c) - r_{\phi}(x, y_r))$$

보상 모델은 데이터에 기록된 선호를 높은 확률로 예측하도록 학습한다. 각 관측이 독립이라고 가정하면, 전체 데이터의 likelihood는 각 선호 확률의 곱이다.

$$\phi^* = \arg \max_{\phi} \prod_{i=1}^N P_{\phi}(y_{c,i} \succ y_{r,i} \mid x_i)$$

로그를 취하면 곱이 합으로 바뀐다. 로그는 단조 증가 함수이므로 최대화하는 파라미터는 동일하다.

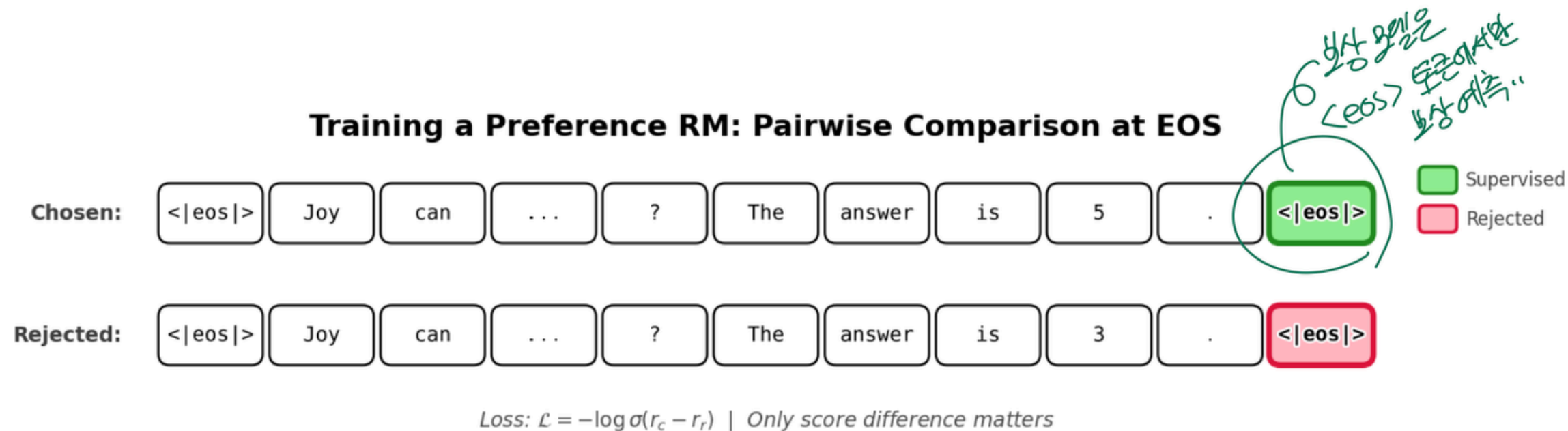
$$\phi^* = \arg \max_{\phi} \sum_{i=1}^N \log \sigma(r_{\phi}(x_i, y_{c,i}) - r_{\phi}(x_i, y_{r,i}))$$

이를 최소화 문제로 바꾸면, 평균 negative log-likelihood가 보상 모델의 loss가 된다.

$$\mathcal{L}_{\text{RM}}(\phi) = -\frac{1}{N} \sum_{i=1}^N \log \sigma(r_{\phi}(x_i, y_{c,i}) - r_{\phi}(x_i, y_{r,i}))$$

Reward Model Architecture

The model computes a scalar score at the EOS token for each completion. The contrastive loss depends only on the **score difference** between chosen and rejected.



기본 구조:

$$(x, y) \xrightarrow{\text{LM}} h_{\text{last}} \xrightarrow{\text{Linear}} r_{\phi}(x, y)$$

- LM의 마지막 유효 토큰 representation을 가져온다.
- scalar head로 응답 전체의 점수를 계산한다.
- chosen과 rejected에 같은 모델을 적용한다.
- 두 점수의 차이로 loss를 계산한다.

Given the model above, the training loss is just three lines:

```
rewards_chosen = model(**inputs_chosen)
rewards_rejected = model(**inputs_rejected)

loss = -nn.functional.logsigmoid(rewards_chosen - rewards_rejected).mean()
```

Practical note: reward models are typically trained for only **1 epoch** to avoid overfitting to the preference data.

Part2.5 Reward model variants

Part 3. Rejection Sampling

Preference RM, ORM, and PRM

모델	평가 대상	학습 데이터	출력
Preference RM	응답 전체의 선호 품질	chosen/rejected pair	응답당 scalar
ORM	최종 정답 여부	응답별 correct/incorrect label	각 응답 토큰에서 정답 확률
PRM	중간 추론 단계의 타당성	단계별 label	step boundary에서 평가

- IFT 데이터 선택에 활용되는 정보 ...

ORM vs. Value Function

겉모습은 비슷해도 예측 대상이 다름

ORM: $P(\text{final outcome is correct} \mid x, y_{\leq t})$

$$\text{Value function: } V^{\pi}(s_t) = \mathbb{E}_{\pi} \left[\sum_{k=t}^T \gamma^{k-t} r_k \mid s_t \right]$$

구분	ORM	Value function
예측 대상	최종 정답 여부	정책을 따를 때의 미래 return
학습 신호	outcome label	rollout에서 추정된 return
용도	filtering, reranking 등	advantage 계산 등

LLM-as-a-Judge and Evaluator Limitations

- 별도의 scalar reward head를 학습하는 대신, LLM에게 응답을 평가하도록 요청할 수 있다.
- pairwise 비교, 점수 부여, 정답 검증 등의 형태로 사용할 수 있다.
- 평가와 데이터 filtering에 활용할 수 있다.
- 평가 모델 자체의 오류와 편향이 결과에 영향을 준다.

Part3. Rejection Sampling

같은 Reward Model이 있어도

RM + Rejection Sampling → 좋은 데이터 선택 후 SFT로 쓸수도 있고

RM + PPO 등의 RL → reward를 직접 최적화로도 쓸 수 있음

Rejection Sampling — Reward-Guided Data Selection

- 1.prompt마다 여러 응답을 생성한다.
 - 2.reward model로 각 응답을 평가한다.
 - 3.높은 점수의 응답을 선택한다.
 - 4.선택한 응답으로 **SFT를 수행**한다.
- | 보상 점수로 학습 데이터를 선별하고, 선택된 응답에 supervised learning을 적용한다.

$$y_{i,j} \sim \pi_{\text{old}}(\cdot \mid x_i)$$

$$j_i^* = \arg \max_j r_\phi(x_i, y_{i,j})$$

$$D_{\text{selected}} = \{(x_i, y_{i,j_i^*})\}_{i=1}^M$$

Selection Strategies — Per-Prompt vs. Global Top-K

방법	선택 기준	장점	주의점
Per-prompt selection	각 prompt의 최고 점수 응답	모든 prompt가 학습에 포함됨	낮은 품질의 응답도 선택될 수 있음
Global top-K	전체 응답 중 점수 상위 K개	높은 점수의 응답에 집중	일부 prompt가 제외될 수 있음

- global selection은 쉬운 prompt에 데이터가 몰릴 수 있고,
- 서로 다른 prompt의 reward score를 직접 비교할 수 있는지도 고려할 필요 있음

Best-of-N vs. Rejection Sampling

- 후보 생성과 평가까지는 동일
- 선택 이후의 사용 방식에서의 차이!
- Best-of-N은 inference time technique
 - Rejection sampling은 Best completion → training data → SFT
 - Best-of-N은 Best completion → 그냥 최종 output으로 사용

구분	Best-of-N	Rejection sampling fine-tuning
후보 생성·평가	수행	수행
선택한 응답 사용	바로 반환	학습 데이터로 사용
모델 파라미터 변경	없음	있음
주요 목적	추론 시 답변 개선	이후 생성 모델 개선

$$y^* = \arg \max_{j \in \{1, \dots, N\}} r_\phi(x, y_j)$$

Take-Home Message

SFT는 좋은 응답을 따라 배우고
reward modeling은 응답의 선호를 점수화하며
rejection sampling은 그 점수로 다음 학습 데이터를 선택한다... SFT

이어지는 PPO 등의 RLHF 기법들은 reward model로
직접 LM의 정책 최적화

*LM의 정책 : 주어진 문맥에서 다음 토큰을 어떤 확률로 선택할지

Thank You